

Arquitectura de Bajo Nivel: CPU y Memoria

Este documento explica de forma profunda cómo funciona el hardware a bajo nivel, centrándose en la CPU (registros, ejecución, flujo) y la memoria RAM (direcciones, stack, heap, jerarquía y relación con la CPU).

PARTE I — CPU Y REGISTROS

1. Qué es una CPU

La CPU (Central Processing Unit) es el componente que ejecuta instrucciones. No entiende variables, funciones ni tipos. Solo entiende instrucciones binarias que operan sobre registros y memoria.

2. Qué es un registro

Un registro es una pequeña celda de almacenamiento dentro de la CPU. Es el nivel más rápido de almacenamiento del sistema y tiene tamaño fijo (64 bits en arquitecturas x86-64).

3. Registros generales en x86-64

Registros como `rax`, `rbx`, `rcx`, `rdx`, `rsi` y `rdi` son registros de propósito general. Aunque históricamente tenían roles específicos, hoy el compilador puede usarlos libremente, respetando ciertas convenciones.

4. `rax` — Registro acumulador

`rax` se utiliza comúnmente para almacenar resultados de operaciones y valores de retorno de funciones. Muchas instrucciones asumen implícitamente el uso de `rax`.

5. `rbx` — Registro preservado

`rbx` es un registro que debe conservar su valor entre llamadas a funciones. Si una función lo modifica, debe restaurarlo antes de retornar.

6. `rcx` — Contador

`rcx` se usa tradicionalmente como contador en bucles y ciertas instrucciones especializadas como desplazamientos y repeticiones.

7. `rdx` — Registro de datos

`rdx` se utiliza junto con `rax` para operaciones aritméticas extendidas, como multiplicaciones y divisiones de gran tamaño.

8. `rsi` y `rdi` — Índices de memoria

`rsi` (source) y `rdi` (destination) se usan para apuntar a direcciones de memoria. También son usados para pasar argumentos a funciones según la convención de llamada.

9. Convención de llamadas

En sistemas Linux x86-64, los primeros argumentos de una función se pasan en registros (`rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`) y el valor de retorno se entrega en `rax`.

10. Registros vs variables

Las variables no existen a nivel de hardware. El compilador decide qué valores viven en registros, cuáles en memoria y por cuánto tiempo.

PARTE II — MEMORIA RAM Y ESPACIO DE DIRECCIONES

1. Qué es la memoria

La memoria es una secuencia lineal de bytes direccionables. No existen tipos ni estructuras a nivel de hardware, solo direcciones y valores.

2. Direcciones de memoria

Cada byte tiene una dirección única. En arquitecturas de 64 bits, las direcciones pueden representar un espacio virtual enorme.

3. Memoria virtual

Cada proceso ve su propio espacio de direcciones virtual. El sistema operativo traduce estas direcciones a memoria física.

4. Mapa típico de memoria de un proceso

Un proceso suele dividir su memoria en secciones: código, datos, heap y stack.

5. Segmento de código

Contiene las instrucciones ejecutables. Generalmente es de solo lectura y compartido entre procesos.

6. Segmentos de datos (DATA y BSS)

Aquí viven variables globales y estáticas. DATA contiene valores inicializados y BSS valores inicializados a cero.

7. Heap

El heap es memoria dinámica usada para estructuras cuyo tamaño o duración no se conoce en tiempo de compilación. Requiere un allocator.

8. Stack

El stack contiene variables locales, argumentos y direcciones de retorno. Cada llamada a función crea un stack frame.

9. Stack frame

Un stack frame define el contexto de una función: variables locales, argumentos y el punto de retorno.

10. Registros y memoria

Los registros suelen contener datos o direcciones que apuntan a memoria. Las instrucciones mueven datos entre registros y memoria constantemente.

11. Jerarquía de memoria

Registros, caché (L1, L2, L3), RAM y almacenamiento secundario forman una jerarquía donde cada nivel es más grande pero más lento.

12. Caché

La caché almacena datos usados recientemente para evitar accesos lentos a RAM. El acceso eficiente a caché es clave para el rendimiento.

13. Relación con compiladores modernos

Compiladores como LLVM analizan el tiempo de vida de los valores para mantenerlos en registros y minimizar accesos a memoria.

14. Rust y el modelo de memoria

Rust impone reglas de ownership y aliasing que permiten al compilador optimizar agresivamente sin comportamiento indefinido.

15. Modelo mental final

Todo se reduce a registros, memoria y saltos. Las abstracciones de alto nivel existen solo para facilitar el razonamiento humano.